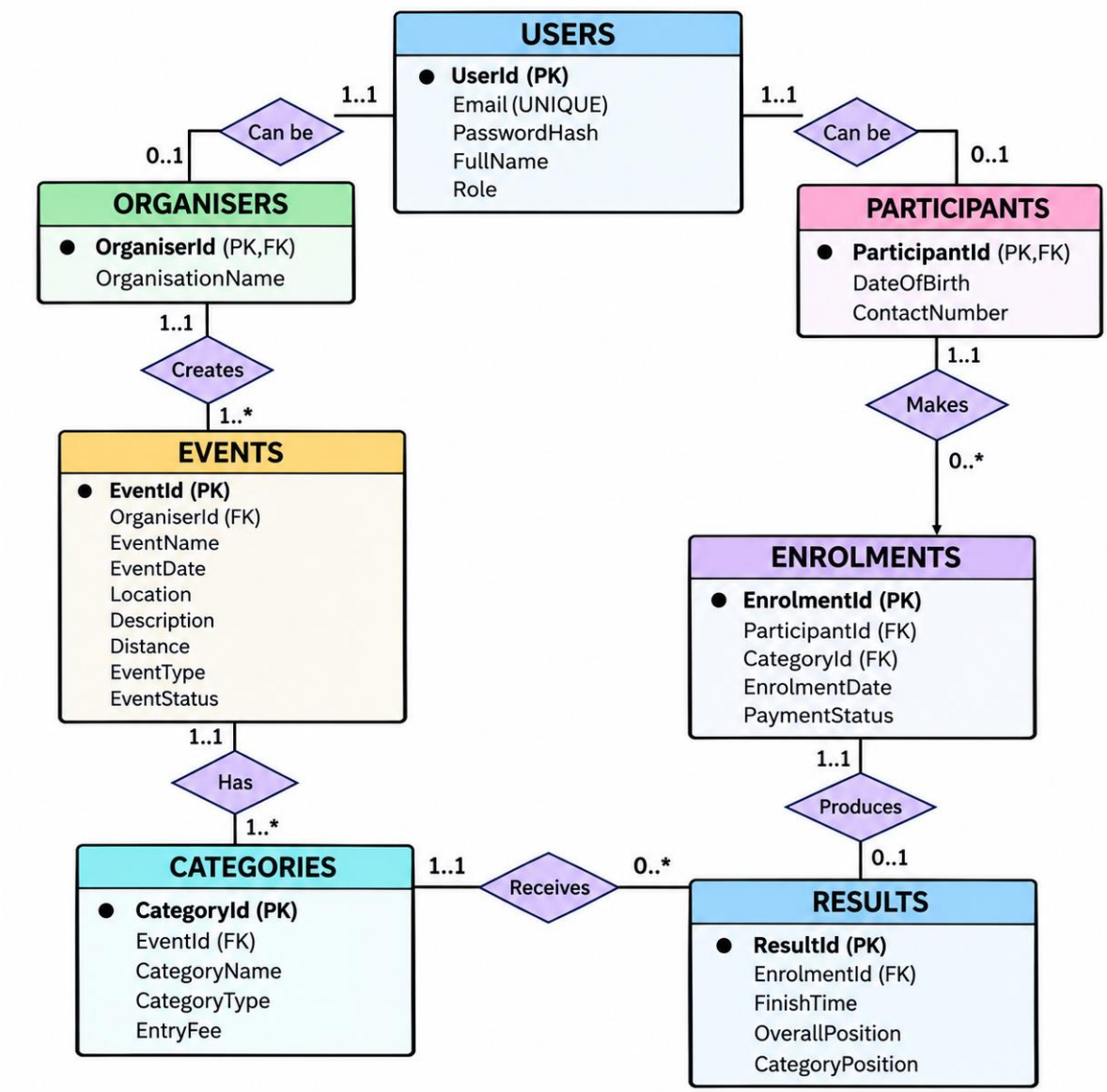
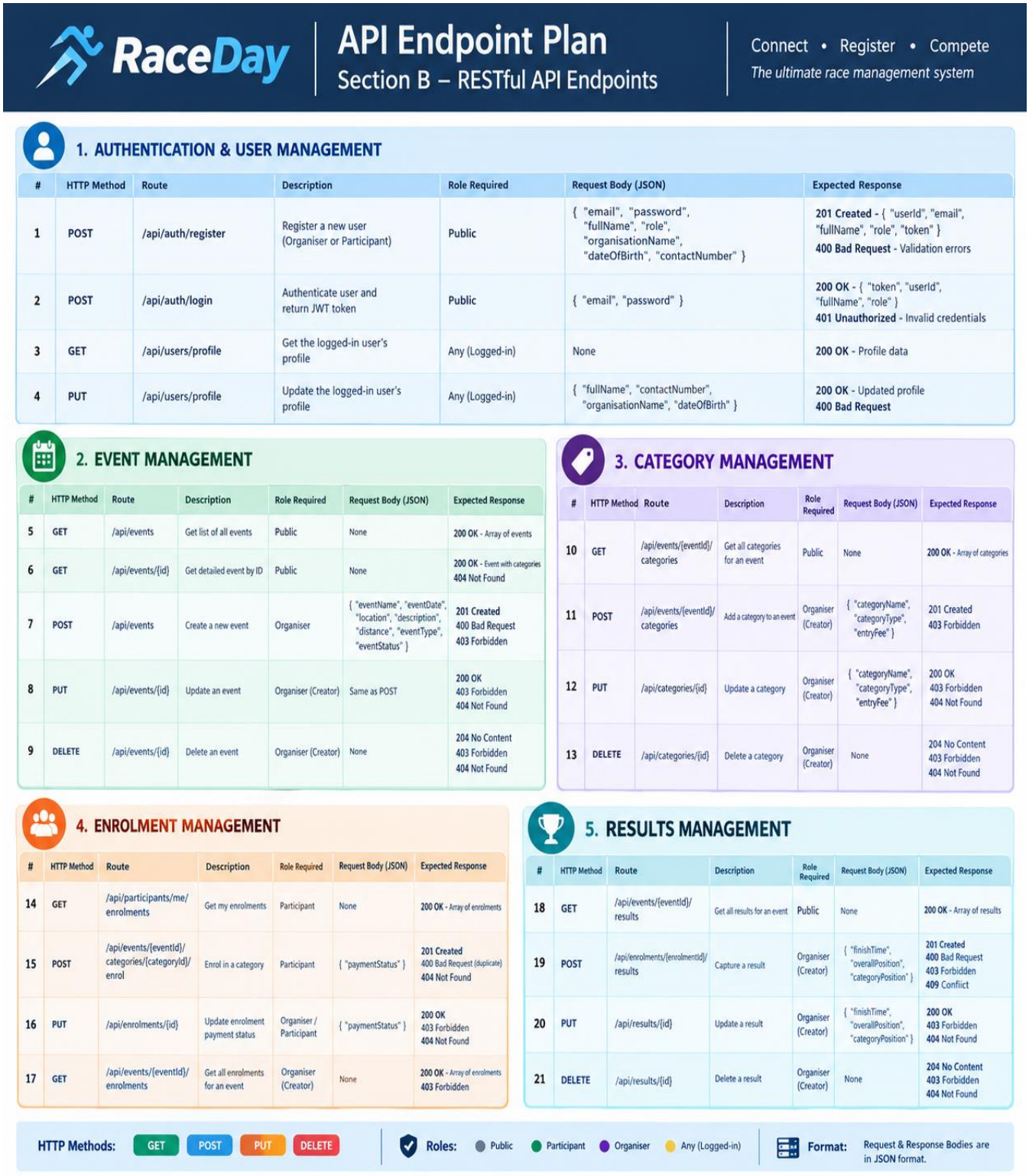
Name: Pontsho Sedumedi
Student Number: ST10483202
Module: Programming 2B
Module Code: PROG6212
Due Date: 16 October 2026

Part 1

Section A: ERD (updated version)



Section B: API Endpoint Plan (updated version)



Section C: SQL Script (updated version)

```
-- ============================================
-- Database: RaceDay
-- Author: Pontsho Sedumedi (ST10483202)
-- Description: Creates the full database schema for the RaceDay system
-- ============================================

-- ============================================
-- STEP 1: Clean Up (Run this section FIRST, separately)
-- ============================================

USE master;

IF DB_ID('RaceDay') IS NOT NULL
BEGIN
   ALTER DATABASE RaceDay SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
   DROP DATABASE RaceDay;
END

-- ============================================
-- STEP 2: Create Database (Run this section SECOND, separately)
-- ============================================

CREATE DATABASE RaceDay;

-- ============================================
-- STEP 3: Use Database (Run this section THIRD, separately)
```

```
-- ============================================

USE RaceDay;

-- ============================================
-- STEP 4: Create Tables (Run this section FOURTH, separately)
-- ============================================

-- 4.1 Users
CREATE TABLE Users (
   UserId INT IDENTITY(1,1) PRIMARY KEY,
   Email NVARCHAR(256) NOT NULL UNIQUE,
   PasswordHash NVARCHAR(256) NOT NULL,
   FullName NVARCHAR(100) NOT NULL,
   Role NVARCHAR(20) NOT NULL CHECK (Role IN ('Organiser', 'Participant'))
);

-- 4.2 Organisers (depends on Users)
CREATE TABLE Organisers (
   OrganiserId INT PRIMARY KEY,
   OrganisationName NVARCHAR(100),
   CONSTRAINT FK_Organisers_Users FOREIGN KEY (OrganiserId)
      REFERENCES Users(UserId) ON DELETE CASCADE
);

-- 4.3 Participants (depends on Users)
CREATE TABLE Participants (
   ParticipantId INT PRIMARY KEY,
```

```
    DateOfBirth DATE,
    ContactNumber NVARCHAR(20),
    CONSTRAINT FK_Participants_Users FOREIGN KEY (ParticipantId)
        REFERENCES Users(UserId) ON DELETE CASCADE
);

-- 4.4 Events (depends on Organisers)
CREATE TABLE Events (
    EventId INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserId INT NOT NULL,
    EventName NVARCHAR(100) NOT NULL,
    EventDate DATETIME NOT NULL,
    Location NVARCHAR(200) NOT NULL,
    Description NVARCHAR(MAX),
    Distance NVARCHAR(50),
    EventType NVARCHAR(20) NOT NULL DEFAULT 'Run'
        CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
    EventStatus NVARCHAR(20) DEFAULT 'Draft'
        CHECK (EventStatus IN ('Draft', 'Open', 'Closed', 'Finished')),
    CONSTRAINT FK_Events_Organisers FOREIGN KEY (OrganiserId)
        REFERENCES Organisers(OrganiserId) ON DELETE CASCADE
);

-- 4.5 Categories (depends on Events)
CREATE TABLE Categories (
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    EventId INT NOT NULL,
    CategoryName NVARCHAR(50) NOT NULL,
```

```
    CategoryType NVARCHAR(20) NOT NULL DEFAULT 'Distance'
        CHECK (CategoryType IN ('Age', 'Distance')),
    EntryFee DECIMAL(10, 2) NOT NULL,
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventId)
        REFERENCES Events(EventId) ON DELETE CASCADE
);

-- 4.6 Enrolments (depends on Participants AND Categories)
-- KEY: FK to Categories uses NO ACTION to prevent multiple cascade path error
CREATE TABLE Enrolments (
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    CategoryId INT NOT NULL,
    EnrolmentDate DATETIME DEFAULT GETDATE(),
    PaymentStatus NVARCHAR(20) DEFAULT 'Pending'
        CHECK (PaymentStatus IN ('Pending', 'Paid')),
    CONSTRAINT FK_Enrolments_Participants FOREIGN KEY (ParticipantId)
        REFERENCES Participants(ParticipantId) ON DELETE CASCADE,
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId)
        REFERENCES Categories(CategoryId) ON DELETE NO ACTION,
    CONSTRAINT UQ_Enrolment_Unique UNIQUE (ParticipantId, CategoryId)
);

-- 4.7 Results (depends on Enrolments)
CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentId INT NOT NULL UNIQUE,
    FinishTime TIME(3),
```

```
    OverallPosition INT,
    CategoryPosition INT,
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId)
        REFERENCES Enrolments(EnrolmentId) ON DELETE CASCADE
);

-- ============================================
-- STEP 5: Insert Sample Data (Run this section FIFTH, separately)
-- ============================================

-- 5.1 Insert Users
INSERT INTO Users (Email, PasswordHash, FullName, Role) VALUES
('john.doe@raceday.com', 'hashed_password_1', 'John Doe', 'Organiser'),
('jane.smith@raceday.com', 'hashed_password_2', 'Jane Smith', 'Organiser'),
('alice.runner@example.com', 'hashed_password_3', 'Alice Runner', 'Participant'),
('bob.walker@example.com', 'hashed_password_4', 'Bob Walker', 'Participant');

-- 5.2 Insert Organisers
INSERT INTO Organisers (OrganiserId, OrganisationName) VALUES
(1, 'Durban Running Club'),
(2, 'Cape Town Cycle Events');

-- 5.3 Insert Participants
INSERT INTO Participants (ParticipantId, DateOfBirth, ContactNumber) VALUES
(3, '1990-05-15', '0821234567'),
(4, '1985-11-20', '0839876543');

-- 5.4 Insert Events
```

```
INSERT INTO Events (OrganiserId, EventName, EventDate, Location, Description, Distance, EventType, EventStatus) VALUES
(1, 'Durban City Marathon', '2026-10-15 06:00:00', 'Durban, KZN', 'A scenic marathon along the Durban coastline.', '42.2km', 'Run', 'Open'),
(1, 'Durban 10km Walk', '2026-11-05 08:00:00', 'Durban, KZN', 'A family-friendly 10km walk through the city.', '10km', 'Walk', 'Open'),
(2, 'Cape Town Cycle Tour', '2026-03-08 07:00:00', 'Cape Town, WC', 'The world-famous Argus Cycle Tour.', '109km', 'Cycle', 'Open');

-- 5.5 Insert Categories
INSERT INTO Categories (EventId, CategoryName, CategoryType, EntryFee) VALUES
(1, 'Full Marathon (42.2km)', 'Distance', 250.00),
(1, 'Half Marathon (21.1km)', 'Distance', 150.00),
(1, 'Senior Men (40+)', 'Age', 200.00),
(2, '10km Open Walk', 'Distance', 50.00),
(2, 'Junior Walk (Under 20)', 'Age', 30.00),
(3, 'Elite Cyclist', 'Age', 400.00),
(3, 'Amateur Cyclist', 'Distance', 300.00);

-- 5.6 Insert Enrolments
INSERT INTO Enrolments (ParticipantId, CategoryId, PaymentStatus) VALUES
(3, 1, 'Paid'),
(3, 4, 'Pending'),
(4, 7, 'Paid');

-- 5.7 Insert Results
INSERT INTO Results (EnrolmentId, FinishTime, OverallPosition, CategoryPosition) VALUES
(1, '03:45:30.123', 15, 2),
```

```
(3, '04:12:15.000', 78, 4);

-- ============================================
-- STEP 6: Verification - Show All Populated Tables
-- ============================================

PRINT '========== USERS ==========';
SELECT * FROM Users;

PRINT '========== ORGANISERS ==========';
SELECT * FROM Organisers;

PRINT '========== PARTICIPANTS ==========';
SELECT * FROM Participants;

PRINT '========== EVENTS ==========';
SELECT * FROM Events;

PRINT '========== CATEGORIES ==========';
SELECT * FROM Categories;

PRINT '========== ENROLMENTS ==========';
SELECT * FROM Enrolments;

PRINT '========== RESULTS ==========';
SELECT * FROM Results;

-- ============================================
```

```
-- STEP 7: Relationship Verification Queries
-- ==========================================

PRINT '========== EVENTS WITH ORGANISERS ==========';
SELECT
    e.EventId,
    e.EventName,
    e.EventDate,
    e.Location,
    e.Distance,
    e.EventType,
    e.EventStatus,
    u.FullName AS OrganiserName,
    o.OrganisationName
FROM Events e
JOIN Organisers o ON e.OrganiserId = o.OrganiserId
JOIN Users u ON o.OrganiserId = u.UserId;

PRINT '========== CATEGORIES WITH EVENTS ==========';
SELECT
    c.CategoryId,
    c.CategoryName,
    c.CategoryType,
    c.EntryFee,
    e.EventName
FROM Categories c
JOIN Events e ON c.EventId = e.EventId;
```

```
PRINT '========== PARTICIPANT ENROLMENTS (Full Details) ==========';
SELECT
   en.EnrolmentId,
   u.FullName AS ParticipantName,
   e.EventName,
   c.CategoryName,
   c.CategoryType,
   en.EnrolmentDate,
   en.PaymentStatus
FROM Enrolments en
JOIN Participants p ON en.ParticipantId = p.ParticipantId
JOIN Users u ON p.ParticipantId = u.UserId
JOIN Categories c ON en.CategoryId = c.CategoryId
JOIN Events e ON c.EventId = e.EventId;

PRINT '========== RESULTS WITH PARTICIPANTS ==========';
SELECT
   r.ResultId,
   u.FullName AS ParticipantName,
   c.CategoryName,
   r.FinishTime,
   r.OverallPosition,
   r.CategoryPosition
FROM Results r
JOIN Enrolments en ON r.EnrolmentId = en.EnrolmentId
JOIN Participants p ON en.ParticipantId = p.ParticipantId
JOIN Users u ON p.ParticipantId = u.UserId
JOIN Categories c ON en.CategoryId = c.CategoryId;
```

```
PRINT '========== TABLE ROW COUNTS ==========';
SELECT 'Users' AS TableName, COUNT(*) AS STRING FROM Users
UNION ALL SELECT 'Organisers', COUNT(*) FROM Organisers
UNION ALL SELECT 'Participants', COUNT(*) FROM Participants
UNION ALL SELECT 'Events', COUNT(*) FROM Events
UNION ALL SELECT 'Categories', COUNT(*) FROM Categories
UNION ALL SELECT 'Enrolments', COUNT(*) FROM Enrolments
UNION ALL SELECT 'Results', COUNT(*) FROM Results;
```

ReadMe File for Part 1 (updated)

📋 Overview

RaceDay is a comprehensive web-based event management platform designed specifically for the South African road running, walking, and cycling community. The system bridges the gap between event organisers and participants by replacing outdated paper-based registrations, spreadsheets, and disconnected communication channels with a modern, integrated digital solution.

From iconic events like the Comrades Marathon and Cape Town Cycle Tour to community park runs and charity cycling events, RaceDay streamlines the entire event lifecycle - from creation and registration to result management and participant tracking.

👥 User Roles

The system supports two distinct user roles with specific permissions:

🏢 Organiser

Event organisers have full control over their events and can:

- Create, edit, and delete events
- Define age or distance categories for each event
- Capture and update participant results (finish times and positions)
- View all enrolments for their events
- Update participant payment statuses
- Manage event status (Draft, Open, Closed, Finished)

🏃 Participant

Race participants can engage with the platform through:

- Account creation and profile management
- Browsing upcoming events
- Entering events by selecting a category
- Viewing their enrolment history
- Tracking personal performance results

- Checking event details including date, location, distance, and type

---

📁 Project Structure

text

```
RaceDay/
├── docs/
│   ├── RaceDay_ERD.png            # Entity Relationship Diagram
│   ├── endpoint-plan.md           # API Endpoint Plan
│   └── RaceDay_Database_Creation.sql  # SQL Database Script
├── .github/workflows/
│   └── validate-docs.yml          # CI/CD Validation
├── README.md
└── .gitignore
```

📊 Part 1 Deliverables

1. Entity Relationship Diagram (ERD)

File: docs/RaceDay_ERD.png

The ERD provides a visual blueprint of the database structure showing all 7 entities, their attributes, and the relationships between them.

Entity	Description
Users	Base table storing authentication and profile information for all users
Organisers	Extends Users with organisation-specific details
Participants	Extends Users with participant-specific details
Events	Stores event information including date, location, distance, type, and status

Entity	Description
Categories	Defines age or distance race options within an event
Enrolments	Records when a participant signs up for a specific category
Results	Stores race results (finish time and positions) linked to enrolments

Key Features:

- Clear primary and foreign key indicators
- Cardinality for all relationships (one-to-many, one-to-one)
- All attributes for each entity
- User role inheritance (Organiser and Participant from User base)

---

2. API Endpoint Plan

File: docs/endpoint-plan.md

The API endpoint plan provides a comprehensive table of all 21 RESTful endpoints the system will expose. This serves as the blueprint for Part 2 implementation.

Category	Purpose	Endpoints
Authentication	User registration and login	2
Profile	View and update user profiles	2
Events	CRUD operations for events	5
Categories	Manage event categories	4
Enrolments	Participant registration and management	4
Results	Capture and manage race results	4

Category	Purpose	Endpoints
Total		**21**

Specifications per endpoint:

- HTTP Method (GET, POST, PUT, DELETE)
- Route (e.g., /api/events, /api/auth/login)
- Description of functionality
- Required Role (Public, Participant, Organiser)
- Request Body (JSON format with examples)
- Expected Response (success and failure scenarios with HTTP status codes)

---

3. SQL Database Script

File: docs/RaceDay_Database_Creation.sql

The SQL script creates the complete database schema in SQL Server Management Studio (SSMS). It includes all tables, constraints, and sample data needed to test the system.

Includes:

- CREATE TABLE statements for all 7 entities
- Primary keys for unique identification
- Foreign keys for relationships between tables
- Constraints (NOT NULL, UNIQUE, CHECK, DEFAULT)
- Referential integrity with cascade deletes on dependent tables

Sample Data:

- 2 Organisers with organisation details
- 2 Participants with personal information
- 3 Events (Durban City Marathon, Durban 10km Walk, Cape Town Cycle Tour)
- 7 Categories covering both age and distance types
- 3 Sample enrolments with payment statuses

- 2 Example results with finish times and positions

Testing: The script runs without errors on a clean SQL Server instance and includes verification queries at the bottom showing all populated tables and joined views.

⚠️ Deviations from Original Part 1 Submission

During the review of Part 1, several design issues were identified in the original ERD that would have prevented Part 2 from meeting the brief's functional requirements. The following changes were made to ensure consistency across all parts.

Changes Made:

Entity	Change	Reason
Organisers	Removed EventId	An Organiser creates many Events; the origi relationship direction was reversed
Participants	Removed EventId	Participants enrol in Categories via Enrolme direct Event linkage was incorrect
Events	Removed CategoryId; added Distance and EventType	One Event has many Categories; Part 2 brief requires distance and event type fields
Categories	Removed UserId; added EventId and CategoryType	Categories belong to Events; Part 2 brief req Age OR Distance category types
Enrolments	Added CategoryId	Participants must select a category when en (required by Part 2 brief)
Results	Removed OrganiserId	Results link through Enrolments; direct Orga linkage was redundant
Assignments	Removed table entirely	Not required for Part 2 functionality; simplifi schema

Technical Note on Cascade Behaviour:

FK_Enrolments_Categories uses NO ACTION (instead of CASCADE) to prevent SQL Server's "multiple cascade paths" error. This means deleting a Category with active Enrolments will fail — API logic handles this by validating before deletion.

Impact:

- The ERD, SQL script, and endpoint plan in /docs have been updated to reflect these changes
- All Part 1 documents are now consistent with the Part 2 implementation
- The core entities (Users, Organisers, Participants, Events, Categories, Enrolments, Results) remain the same

---

✅ CI/CD Pipeline

Workflow: .github/workflows/validate-docs.yml

The GitHub Actions workflow automatically validates the repository structure on every push and pull request, ensuring all required documents are present.

Validation Checks:

- ✅ /docs folder exists
- ✅ ERD image (PNG or PDF) is present
- ✅ Endpoint plan (Markdown) is present
- ✅ SQL script (SQL) is present

Green Build Status:

https://docs/cicd-green-build.png

Screenshot of successful GitHub Actions build

🚀 Setup Instructions

Prerequisites

- SQL Server Management Studio (SSMS)
- Git
- GitHub Account
- (For Part 2) Visual Studio 2022 with .NET 8 SDK

Steps

1. **Clone the Repository**

bash

git clone https://github.com/Rea-0611/RaceDay-Prog-2B-POE.git

cd RaceDay-Prog-2B-POE

2. **Review Planning Documents**
    - Open docs/RaceDay_ERD.png to understand the database structure
    - Read docs/endpoint-plan.md to understand the API design
    - Examine docs/RaceDay_Database_Creation.sql for database implementation
3. **Run the Database Script**
    - Open SQL Server Management Studio
    - Connect to your SQL Server instance
    - Open docs/RaceDay_Database_Creation.sql
    - Execute Step 1 (Cleanup) against master database
    - Execute Steps 2-7 sequentially
    - Verify no errors occurred
4. **Verify Database Creation**

sql

USE RaceDay;

-- Check row counts

SELECT 'Users' AS TableName, COUNT(*) AS RowCount FROM Users

UNION ALL SELECT 'Organisers', COUNT(*) FROM Organisers

UNION ALL SELECT 'Participants', COUNT(*) FROM Participants

UNION ALL SELECT 'Events', COUNT(*) FROM Events

UNION ALL SELECT 'Categories', COUNT(*) FROM Categories

UNION ALL SELECT 'Enrolments', COUNT(*) FROM Enrolments

UNION ALL SELECT 'Results', COUNT(*) FROM Results;

Expected: 4 Users, 2 Organisers, 2 Participants, 3 Events, 7 Categories, 3 Enrolments, 2 Results